IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2023

DISCRETE MATHEMATICS

Wednesday 17 May 14:30

Time allowed: 2:00 hours

Answer ALL questions.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : D. Goodman
Second Marker(s) :

© Imperial College London

DISCRETE MATHEMATICS

1. a) For each of the following statements, state whether it is true or false and provide a supporting proof.

   i) $\max\{100n^2, n^3\} = O(n^3)$. [3]

   ii) $\min\{100n^2, n^3\} = \Omega(n^3)$. [3]

   iii) $n^3 \sin(n\pi/15) = \Theta(n^3)$. [3]

   b) Give a tight bound for each of the following recurrence relations, or explain why it is not possible to do so.

   i) $T(n) = 8T(n/4) + O(n\sqrt{n})$. [2]

   ii) $T(n) = 9T(n/3) + n^2 + n$. [2]

   iii) $T(n) = 4T(n/4) + 2n$. [2]

Master Theorem. If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) \;+\; O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

2. The quicksort algorithm sorts an array X of n elements in the following way. The last item in the array X_{n-1} is called the *pivot*. The algorithm scans through the first $n-1$ items of the array dividing them into those less than or equal to the pivot value, and those larger than it. It then recursively sorts those two subarrays. Finally, it joins the sorted subarrays with the pivot value in the middle. In pseudocode:

```
function QUICKSORT(X)
    n = len(X)
    if n ≤ 1 then
        return X
    end if
    p = X_{n-1}
    L = array of elements X_i where X_i ≤ p and 0 ≤ i < n − 1
    R = array of elements X_i where X_i > p and 0 ≤ i < n − 1
    L = QUICKSORT(L)
    R = QUICKSORT(R)
    return join arrays L, [p], R
end function
```

Write $T(n)$ for the time it takes to run this code on an array of length n.

a) Let $k = \text{len}(L)$ be the length of the array of elements $X_i \leq p$. Write a recurrence equation for $T(n)$ with terms of the form $T(m)$ for $m < n$ and $\Theta(\cdots)$. [3]

b) If at each iteration, the algorithm divides X into two equal halves L and R, solve the recurrence relationship to find the time complexity of the algorithm. [3]

c) If at each iteration, the pivot $p = X_{n-1}$ is strictly greater than all other values X_i $(0 \leq i < n-1)$, solve the recurrence relationship to find the time complexity of the algorithm. [3]

d) Based on the above, what is the best and worst case performance of this algorithm? You should briefly explain your reasoning, but you do not need to provide a proof. [1]

3. We can represent an *operator* $x \oplus y$ in code by a function `f(x, y)`.

The *reduction* of an operator across an array of values X of size n is

$$X_0 \oplus X_1 \oplus X_2 \oplus \cdots \oplus X_{n-1},$$

or in code

```
f(X[0], f(X[1], f(..., f(X[n-2], X[n-1])...))).
```

You may assume that the operator is *associative* and *commutative*, so that the order of operations does not matter. Associative means $x \oplus (y \oplus z) = (x \oplus y) \oplus z$. Commutative means $x \oplus y = y \oplus x$.

a) Write code to compute the reduction of an operator defined by a function `f(x, y)` across an array X with n elements. Your code should use a loop, not recursion. [2]

b) Compute the time complexity of your code, assuming each call `f(x, y)` is $O(1)$. [2]

c) Define and use a loop invariant to prove that your code is correct. [10]

d) For each of the algorithms below, state whether or not they can be implemented as reductions of an operator. If they can be implemented as reductions, define the operator `f(x, y)`. If not, explain why (you do not need to write a proof).

i) The sum of all values in an array. [2]

ii) The smallest difference between two values in an array:

$$\min_{0 \le i < j < n} |X_i - X_j|$$

[2]

iii) The maximum value in an array. [2]

e) Define and use the reduction of a non-commutative operator to write an algorithm to find the second largest value in an array X.

Hint: the function `f(x, y)` should return a pair of values, `x` should be an element of the array and `y` a pair of values. [5]